

API, Authentication & Prompting

Topics 1 – 6 | Complete Reference Guide

API Basics · Local Variables · Auth Types · OAuth · Prompting Techniques
· Bias & Support Parameters

TOPIC 1

What is an API?

An **API (Application Programming Interface)** is a set of defined rules and protocols that allows different software applications to communicate with each other. It acts as an intermediary — a 'waiter' that takes a request from one system, delivers it to another, and returns the response.

1. Simple Analogy

Think of a restaurant: you (the client) look at the menu and place an order. The waiter (API) takes your order to the kitchen (server/service) and brings back your food (response). You never need to go into the kitchen yourself.

2. Core Components of an API

Component	Description
Endpoint	A specific URL that represents a resource or action. e.g. /api/users
Request	What the client sends — method, headers, body, query params.
Response	What the server returns — status code, headers, data (usually JSON).
Method	HTTP verb: GET (read), POST (create), PUT (update), DELETE (remove).
Headers	Metadata in the request/response — Auth tokens, Content-Type, etc.
Status Code	HTTP code indicating result: 200 OK, 201 Created, 404 Not Found, 500 Error.

3. Types of APIs

Type	Description
REST API	Representational State Transfer. Stateless, uses HTTP. Most widely used today.
SOAP API	Simple Object Access Protocol. XML-based, strict contracts. Used in banking/enterprise.
GraphQL API	Query language for APIs. Client requests exactly the data it needs. Developed by Meta.
WebSocket	Full-duplex real-time communication. Used in chat apps, live dashboards.

gRPC	Google's high-performance RPC framework using Protocol Buffers.
Webhooks	Reverse API — server pushes data to client when an event occurs.

4. How a REST API Call Works

Client Server

```
| |  
|-- GET /api/users/42 --> |  
| Headers: Auth Token |  
| |  
|<-- 200 OK ----- |  
| { "id": 42, |  
| "name": "Alice" } |
```

5. Real-World API Examples

- Google Maps API – Embed maps and get directions in any app.
- Stripe API – Process payments securely without building a payment system.
- OpenAI / Anthropic API – Access AI language models like Claude or GPT.
- Twitter/X API – Read tweets, post updates, analyse trends programmatically.
- Weather API (OpenWeatherMap) – Fetch live temperature, forecast data.
- GitHub API – Manage repositories, issues, pull requests via code.
- Twilio API – Send SMS, make calls, handle phone numbers programmatically.

6. API Request Example (Python)

```
import requests  
  
url = 'https://api.example.com/users/42'  
headers = {'Authorization': 'Bearer YOUR_TOKEN'}  
response = requests.get(url, headers=headers)  
print(response.status_code) # 200  
print(response.json()) # {'id': 42, 'name': 'Alice'}
```

TOPIC 2

Overview of Local Variables

A **local variable** is a variable declared inside a function, block, or scope and is only accessible within that specific block. Once execution exits that block, the variable is destroyed and its memory is freed.

1. Purpose of Local Variables

- Encapsulation – Keeps data confined to where it's needed, reducing side effects.
- Memory Efficiency – Allocated on the call stack; freed immediately after the scope ends.
- Name Safety – Same variable name can exist in multiple functions without conflict.
- Readability – Makes it clear which data belongs to which part of the program.
- Debugging – Limits where a bug can originate, simplifying troubleshooting.

2. Scope and Lifetime

Concept	Meaning
Scope	The region of code where the variable is visible and can be used.
Lifetime	Duration from variable creation (entering the block) to destruction (exiting).
Stack	Local variables live on the call stack — allocated on entry, popped on exit.

3. Local Variables vs Global Variables

Aspect	Local Variable	Global Variable
Scope	Inside function/block only	Entire program / module
Lifetime	Duration of function call	Entire program runtime
Access	Only within declaring scope	Any function can read/modify
Risk	Low – isolated	High – unintended side-effects
Memory	Stack (fast, auto-freed)	Heap/data segment (manual or GC)

4. Examples Across Languages

Python

```
def greet():
```

```
message = 'Hello, World!' # local variable
print(message)
greet()
# print(message) --> NameError: 'message' is not defined
```

JavaScript

```
function calculateArea(r) {
  const pi = 3.14159; // local
  const area = pi * r * r;
  return area;
}
// pi and area not accessible here
```

Java

```
public void printScore() {
  int score = 95; // local
  System.out.println(score);
}
// score not accessible outside method
```

5. Key Usage Rules

- Always initialise local variables before use — uninitialized variables cause errors.
- Keep local variables as narrow in scope as possible (declare inside loops/ifs when applicable).
- Avoid shadowing — naming a local variable the same as an outer-scope variable causes confusion.
- Use local variables for intermediate calculations, loop counters, and temporary data.
- In Python, use the **global** or **nonlocal** keyword explicitly if you need to modify an outer variable.

TOPIC 3

Types of Authentication, Errors & OAuth

Authentication is the process of verifying **who** someone is before granting access to a resource. Different mechanisms offer varying trade-offs between security, usability, and complexity.

1. Types of Authentication

Method	Description
Basic Auth	Username + password sent Base64-encoded in every request header. Simple but insecure without HTTPS.
API Key	A secret token sent in the header or query string. Easy to implement; no user context.
Bearer Token (JWT)	A signed token (JSON Web Token) sent in the Authorization header. Stateless, widely used.
OAuth 2.0	Delegated auth framework. User grants a third-party limited access without sharing passwords.
OpenID Connect	Identity layer on top of OAuth 2.0. Provides user identity (who you are), not just access.
Session / Cookie	Server creates a session after login and sends a session ID via a cookie. Stateful.
SAML	XML-based SSO standard used in enterprises. Browser-based redirects between IdP and SP.
TOTP / MFA	Time-based One-Time Passwords (Google Authenticator) add a second factor of verification.
Biometric	Fingerprint, Face ID — used on devices; backend uses a key derived from the biometric scan.
Certificate (mTLS)	Mutual TLS — both client and server present certificates. Used in microservices/zero-trust.

2. Common Authentication Errors

Error Code	Meaning
400 Bad Request	Malformed request — missing required fields, invalid JSON, wrong content-type.
401 Unauthorized	Not authenticated — no token, expired token, or invalid credentials.

403 Forbidden	Authenticated but not authorised — valid token, insufficient permissions.
404 Not Found	Resource or endpoint doesn't exist.
422 Unprocessable	Request is well-formed but semantically incorrect — validation failure.
429 Too Many Requests	Rate limit exceeded — too many auth attempts.
500 Internal Server Err	Server-side problem during auth processing.
503 Service Unavailable	Auth service (e.g. IdP) temporarily down.

3. Authentication vs Authorisation

Term	Definition
Authentication (AuthN)	Who are you? Verifying identity via credentials.
Authorisation (AuthZ)	What can you do? Granting permissions after identity is confirmed.

4. OAuth 2.0 — Deep Dive

OAuth 2.0 is an **authorisation framework** (not authentication) that allows an application to obtain limited access to a user's account on another service, without the user sharing their password. The user grants the app permission; the OAuth server issues a token.

Key Roles in OAuth 2.0

Role	Description
Resource Owner	The user who owns the data and grants access.
Client	The application requesting access (your app).
Authorization Server	Issues access tokens after user consent (e.g. Google, GitHub).
Resource Server	The API holding protected data — validates access tokens.

OAuth 2.0 Flow (Authorization Code Grant)

1. User clicks 'Login with Google' in your app
2. App redirects --> Google Auth Server
GET /auth?client_id=APP_ID&redirect_uri=...&scope=email
3. User logs in & consents on Google
4. Google redirects back --> your app with an Authorization Code
GET /callback?code=AUTH_CODE
5. Your server exchanges code for tokens:
POST /token { code, client_id, client_secret, redirect_uri }
6. Auth Server returns Access Token (+ Refresh Token)

7. Your app calls resource server using the Access Token

GET /api/profile Authorization: Bearer ACCESS_TOKEN

OAuth 2.0 Grant Types

Grant Type	Use Case
Authorization Code	Most secure. Used by web/mobile apps with a backend server.
PKCE Extension	Authorization Code without client secret — for single-page apps & mobile.
Client Credentials	Machine-to-machine (no user). Service authenticates with its own credentials.
Device Code	For devices with no browser (smart TVs). User enters a code on another device.
Implicit (legacy)	Deprecated — token returned directly in URL. No longer recommended.

Token Types

- **Access Token** – Short-lived (minutes to hours). Used to call protected APIs.
- **Refresh Token** – Long-lived. Used to get a new Access Token without re-login.
- **ID Token (OIDC)** – JWT containing user identity claims (name, email, sub).

Common OAuth Errors

Error Code	Cause
invalid_request	Missing or duplicate parameter in the request.
unauthorized_client	Client is not allowed to use this grant type.
access_denied	User denied permission on the consent screen.
invalid_scope	Requested scope is unknown or exceeds what's allowed.
invalid_grant	Authorization code or refresh token is expired/used already.
invalid_client	Client authentication failed — wrong client_id or secret.
server_error	Auth server encountered an unexpected error.

TOPIC 4

Prompting Techniques

Prompt engineering is the practice of crafting inputs to AI language models to elicit accurate, relevant, and well-structured outputs. The right technique depends on the task complexity and the model being used.

1. Zero-Shot Prompting

Give the model a task with no examples. Relies entirely on the model's pre-trained knowledge.

Prompt: "Translate the following English text to French: 'Good morning'"

Output: "Bonjour"

2. One-Shot Prompting

Provide exactly one example before the actual task. Helps the model understand format/style.

Example: English: "Hello" -> French: "Bonjour"

Now translate: English: "Thank you" -> French: ?

3. Few-Shot Prompting

Provide 2–5 examples demonstrating the desired pattern. Highly effective for format adherence.

Q: Capital of France? A: Paris

Q: Capital of Japan? A: Tokyo

Q: Capital of India? A: ?

4. Chain-of-Thought (CoT) Prompting

Instruct the model to show its reasoning step-by-step before giving the final answer. Dramatically improves accuracy on math, logic, and multi-step reasoning tasks.

Prompt: "Solve step-by-step: A shop sells 12 apples/hour. How many in 7.5 hours?"

Output: Step 1: $12 \times 7 = 84$. Step 2: $12 \times 0.5 = 6$. Total = 90 apples.

5. Self-Consistency Prompting

Generate multiple chains of thought for the same question, then pick the most common answer. Useful for reducing errors on reasoning tasks.

6. Tree of Thoughts (ToT)

Explore multiple reasoning paths simultaneously, like a decision tree. The model evaluates intermediate steps and backtracks if needed. Best for complex planning problems.

7. ReAct Prompting (Reason + Act)

Interleaves reasoning with actions — the model thinks, decides on an action (e.g. search the web, run code), observes the result, and continues reasoning. Foundation of AI agents.

Thought: I need to know today's exchange rate.

Action: search('USD to EUR exchange rate today')

Observation: 1 USD = 0.92 EUR

Thought: Now I can answer the question.

8. Role / Persona Prompting

Assign a role to the model to shape tone, expertise, and style of output.

```
"You are a senior Python developer. Review this code and suggest improvements:"
```

9. Instruction Prompting

Clearly state what you want using action verbs. Be explicit about format, length, and constraints.

```
"Summarise the following article in exactly 3 bullet points. Use simple language."
```

10. Prompt Chaining

Break a complex task into a pipeline of simpler prompts. The output of one prompt becomes the input to the next.

- Step 1: Extract key facts from a document.
- Step 2: Format extracted facts into a structured JSON.
- Step 3: Use JSON to generate a summary report.

TOPIC 5

Existing Shortened Types of Prompts

In practice, prompts are often categorised by their structure and shorthand names that describe their pattern. Below are the most widely referenced prompt types and their concise definitions.

1. Quick Reference — Prompt Types

Type	One-Line Definition	Best For
Zero-Shot	No examples given	Simple Q&A;, translation
One-Shot	One example given	Format demonstration
Few-Shot	2–5 examples given	Classification, extraction
CoT	Chain-of-Thought — show reasoning	Math, logic, multi-step
CoT + Few-Shot	Examples + reasoning steps	Complex reasoning tasks
SC	Self-Consistency — multiple paths	Accuracy-critical tasks
ToT	Tree of Thoughts — branching paths	Planning, puzzles
ReAct	Reasoning + Acting interleaved	AI agents, tool use
APE	Automatic Prompt Engineer — AI writes prompts	Prompt optimisation
OPRO	Optimise prompts via gradient-free search	Research/automation
System Prompt	Background instructions before conversation	Chatbots, assistants
Meta-Prompt	Prompt that generates other prompts	Automation pipelines
Delimiter Prompt	Uses XML/JSON/triple-quotes to structure input	Complex inputs
Negative Prompt	Tells model what NOT to do	Image gen, constrained output

2. Structural Prompt Formats

RISEN Framework

Element	Description
R – Role	Define who the model is: 'You are an expert data scientist.'
I – Input	Provide the data or context to work with.

S – Steps	List the specific steps you want followed.
E – Expectation	Describe the desired output format and quality.
N – Narrowing	Add constraints: length, tone, language, audience.

CO-STAR Framework

Element	Description
C – Context	Background info the model needs.
O – Objective	The specific task or goal.
S – Style	Writing style — formal, casual, technical.
T – Tone	Emotional tone — friendly, serious, persuasive.
A – Audience	Who the output is for — experts, beginners, etc.
R – Response	Desired output format — JSON, markdown, prose.

3. Prompt Templates by Use Case

- Summarisation: 'Summarise the following [text] in [N] sentences for [audience].'
- Classification: 'Classify the sentiment of this review as Positive/Negative/Neutral.'
- Code Gen: 'Write a [language] function that [does X]. Include docstrings and error handling.'
- Extraction: 'Extract all [entities] from the text below and return as a JSON list.'
- Rewriting: 'Rewrite the following paragraph to be more [concise / formal / simple].'
- Q&A: 'Based only on the context below, answer: [question]. Context: [text]'

TOPIC 6

Prompt Resolution, Bias & Support Parameters

Part A – Prompt Resolution

Prompt resolution refers to how a language model interprets, processes, and resolves ambiguities in a given prompt to produce a coherent output. Understanding this helps you write prompts that produce reliable results.

How a Model Resolves a Prompt

Stage	Description
Tokenisation	Input text is split into tokens (subword units). Each token is embedded.
Context Window	All tokens (system prompt + history + user input) must fit in the context window.
Attention	Self-attention layers weigh the importance of each token relative to all others.
Decoding	The model samples or greedily selects the next token based on probability distribution.
Stop Condition	Generation stops at a stop token, max_tokens limit, or user-defined stop sequence.

Common Resolution Issues & Fixes

Issue	Cause & Solution
Ambiguity	Vague prompt → unexpected interpretation. Fix: be explicit and specific.
Prompt Leakage	Model reveals system prompt. Fix: use strong system prompt boundaries.
Instruction Conflict	Contradictory instructions → inconsistent output. Fix: order and prioritise clearly.
Context Loss	Long conversations lose early context. Fix: re-state key info or use summaries.
Hallucination	Model invents false facts. Fix: add 'only use provided context' or grounding.
Format Drift	Output format changes mid-conversation. Fix: reinforce format in every turn.

Part B – Bias Parameters

Bias parameters control how the model distributes probability across possible next tokens. They let you tune creativity, repetitiveness, and focus of the output.

1. Temperature

Controls **randomness** in token selection. Higher = more creative and varied; lower = more deterministic and predictable.

Value	Effect
0.0	Fully deterministic — always picks the highest-probability token. Best for factual Q&A.;
0.3 – 0.5	Low creativity. Good for summarisation, code generation, structured tasks.
0.7 – 0.9	Balanced creativity. Default for most chatbots and general tasks.
1.0 – 1.5	High creativity. Good for brainstorming, fiction writing, poetry.
2.0+	Very random — output often incoherent. Rarely useful in practice.

2. Top-P (Nucleus Sampling)

Instead of sampling from all tokens, restricts sampling to the smallest set of tokens whose cumulative probability exceeds P. Works alongside temperature.

Value	Effect
top_p = 0.1	Very narrow — only the most likely tokens considered. Very focused output.
top_p = 0.5	Moderate variety — top 50% of probability mass sampled.
top_p = 0.9	Wide variety — 90% of probability mass. Default for most models.
top_p = 1.0	All tokens considered (no nucleus filtering).

3. Top-K Sampling

Limits token selection to the top K most probable tokens at each step.

Value	Effect
top_k = 1	Greedy decoding — always the most probable token. Deterministic.
top_k = 10	Only the 10 most probable tokens considered. Very focused.
top_k = 50	50 candidates — moderate diversity.
top_k = 0	Disabled (use top_p instead).

4. Frequency Penalty

Reduces the probability of tokens that have already appeared, based on how often they appeared. Reduces repetition.

Value	Effect
0.0	No penalty. Model may repeat words freely.
0.5	Moderate reduction in already-used words.
1.0	Strong discouragement of repeating any token already in the output.
2.0	Very strong — output may avoid even necessary repetitions.

5. Presence Penalty

Reduces the probability of any token that has appeared at least once, regardless of frequency. Encourages the model to introduce new topics.

Value	Effect
0.0	No effect. Model stays on existing topics.
0.6	Moderate push to introduce variety and new concepts.
1.0	Strong push towards novel content in each turn.

6. Repetition Penalty (Hugging Face models)

A multiplier applied to the logits of already-generated tokens. Values > 1 penalise repetition; values < 1 encourage it.

Value	Effect
1.0	No penalty (default).
1.2	Mild discouragement of repetition.
1.5	Strong anti-repetition. Good for long-form text.

Part C – Support Parameters

Support parameters configure the behaviour, constraints, and context of the API call itself rather than the probability distribution.

Parameter	Purpose
max_tokens	Maximum number of tokens in the response. Prevents runaway output.

stop	A string or list of strings where generation stops. e.g. stop=['\n', '###']
stream	If true, returns tokens incrementally (Server-Sent Events). Enables real-time output.
n	Number of completions to generate per request. Useful for SC prompting.
model	Which model to use: gpt-4o, claude-sonnet-4-6, gemini-2.0-flash, etc.
system	System-level instruction (Anthropic/OpenAI). Sets persona and rules.
messages	Conversation history array of {role, content} pairs.
seed	Integer seed for reproducible outputs (supported by some models).
logit_bias	Map of token_id -> bias value to manually boost or suppress specific tokens.
tools / functions	Define external tools (APIs, code execution) the model can invoke.
tool_choice	Force the model to call a specific tool or let it choose automatically.
response_format	Set to {"type": "json_object"} to enforce JSON output.
context_window	Total token budget for input + output. GPT-4o: 128K, Claude 3.5: 200K.
timeout	Request timeout in seconds — important for long-running completions.

Recommended Parameter Combos by Task

Task	Parameters	Goal
Factual Q&A;	temp=0.1, top_p=0.9, freq_pen=0.0	Deterministic, accurate
Code Generation	temp=0.2, top_p=0.95, freq_pen=0.1	Consistent, minimal repetition
Creative Writing	temp=1.0, top_p=0.95, pres_pen=0.6	High variety, novel content
Summarisation	temp=0.3, top_p=0.9, max_tokens=300	Concise, on-topic
Chatbot	temp=0.7, top_p=0.9, pres_pen=0.3	Balanced, conversational
JSON Extraction	temp=0.0, response_format=json_object	Strict, structured output

